Robots:

Les robots sont stockes dans le package `robots`
Les robots sont instancies par leur `String nomTypeRobot` via la classe a vocation statique `RobotFactory`.

La classe Robot prenant trop de place pour ce compte-rendu fournit le constructeur basique de la classe auxquel toutes les classes filles se referent ainsi que certaines fonctions basique d'utilite generale pour tous les robots ou des utilitaires de calcul statiques.

De sorte que l'ajout d'un nouveau robot se fait aisement: Les particularites de chaque type de robot etant alors gerees dans les sous classes comme celle qui suit.

```
package robots;

import carte.NatureTerrain;
import carte.Position;

public class RobotRoues extends Robot{
    private static final double VITESSE_DEFAUT = 80;
    private static final double VITESSE_MAX = 150;
    private static final int CAPACITE = 5000;
    /**
     * Classe implementant le robot a roues.
     *
     * @param position position initiale du robot
     * @param vitesseFichier vitesse du robot, limitee internalement, fournir valeur
negative pour vitesse par defaut
     * @see Robot
     */
    public RobotRoues(Position position, int vitesseFichier){
        super(position, vitesseFichier, VITESSE_DEFAUT, VITESSE_MAX);
    }

    @Override
    public int getCapacite() {
        return CAPACITE;
    }

    @Override
    public double getVitesse(NatureTerrain t) {
        return  switch (t) {
            case TERRAIN_LIBRE -> getVitesse();
            case HABITAT -> getVitesse();
            default->0; //case non traversable
        };
    }
}
```

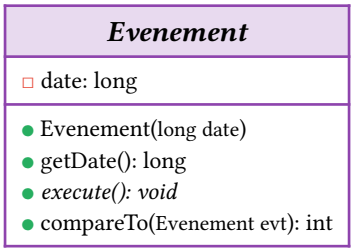
La classe parente implemente ensuite par ex:
`estTraversable(Case case) -> getVitesse(case.nature)>0`

La creation d'un robot via RobotFactory se fait ainsi:

```
//dans l'ordre (String typeRobot, Position p, double vitesse);
//n'importe quelle vitesse negative se voit remplacer par la valeur par defaut
//une vitesse fournie est limitee par le max autorise du robot.
Robot r = RobotFactory.createRobot("ROUES", new Position(0,0), -1);
```

Evenements:

Les evenements sont dans le package `event`:
Tous les evenements descendent d'une classe abstraite Evenement.



L'evenement implemente l'interface `Comparable<Evenement>`
Qui permet son insertion rapide dans le heap d'attente en log(n) et de sortir les elements en O(1).

Carte

Le package `carte` contient tous les elements relatifs a la carte:

Le type `Case` herite de `Position` et lui ajoute un type de case. Ceci permet d'implementer l'indexation dans une `HashTable`: des feux par leur position sans connaitre le type de la case. Mais aussi d'y acceder directement connaissant la case ou l'on est.

Exemple d'utilisation:

```
//on suppose Hashmap<Position, Incendie> incendies; declare et initialise

Incendies inc = incendies.get(new Position(0,0));

//mais aussi si on a:
Case c = carte.getCase(0,0);
System.out.println(c);
Incendie inc = incendies.get(c);

//dans ces deux cas on recupere une valeur nulle si l'incendie n'existe pas
```

toString

Des override pour les methodes toString ont ete faites sur:

```
Carte
Case
DonneesSimulation
```

ceci permet de verifier le bon chargement des valeurs apres lecture.